

北京邮电大学 本科毕业设计（论文）初期进度报告

Project Early-term Progress Report

学院 School	International School	专业 Programme	Intelligent Science and Technology		
姓 Family name	赵 Zhao	名 First Name	哲昀 Zheyun		
BUPT 学号 BUPT number	2022213670	QM 学号 QM number	221170559	班级 Class	2022215124
论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems				

Literature Review

1. Background and Motivation

Large Language Model (LLM)-powered applications are rapidly evolving from single-request inference services into complex **multi-agent systems**, where multiple specialized agents collaborate through structured workflows to solve sophisticated tasks such as question answering, planning, code generation, and retrieval-augmented reasoning. In such systems, a single user request may trigger a sequence of LLM invocations, tool calls, and inter-agent communications, making **end-to-end (E2E) response latency** the primary determinant of user experience.

At the same time, modern LLM services are increasingly deployed in **shared, multi-tenant public cloud environments**, where multiple applications and agents contend for the same underlying inference infrastructure. Under high or bursty loads, resource contention at both the compute level (e.g., GPU queues, memory pressure) and the communication level (e.g., network latency, cross-node traffic) can significantly amplify queueing delays and tail latency. These challenges motivate a growing body of research on **end-to-end optimization of LLM-based applications**, shifting focus from isolated inference efficiency toward system-level coordination across workflows, resources, and execution environments.

This literature review surveys recent representative work in this area, focusing on five closely related research directions: (1) workflow-aware multi-agent serving under shared resources, (2) application semantic exposure for end-to-end optimization, (3) inference-stage disaggregation for improved service goodput, (4) fine-grained workflow orchestration frameworks, and (5) online network latency estimation using exponentially weighted moving averages (EWMA). By analyzing the strengths and limitations of existing approaches, this review identifies a clear research gap at the intersection of **multi-agent workflow scheduling and dynamic network-aware resource management**, which motivates further investigation.

2. Workflow-Aware Multi-Agent Serving under Shared LLM Resources

As multi-agent applications increasingly rely on shared LLM backends, a key challenge lies in coordinating heterogeneous agent workloads under resource contention. Kairos directly addresses this problem by targeting **low-latency multi-agent serving with shared LLMs under excessive loads** [1]. The central observation of Kairos is that different agents within the same application exhibit substantially different inference characteristics, including prompt lengths, output lengths, latency distributions, and GPU memory demands. However, existing serving systems often schedule requests in a first-come-first-served (FCFS) manner and distribute them across instances without considering such inter-agent differences, leading to inefficient queueing and increased tail latency.

To address this issue, Kairos introduces a workflow-aware serving architecture composed of three main components: an online workflow orchestrator that reconstructs dynamic agent workflows at runtime, a priority scheduler that orders requests based on agent-specific remaining latency distributions, and a memory-aware dispatcher that assigns requests to LLM instances while avoiding GPU memory overcommitment [1]. By explicitly incorporating workflow structure and

agent heterogeneity into the scheduling loop, Kairos demonstrates significant reductions in end-to-end latency compared to prior end-to-end optimization systems.

Despite these contributions, Kairos primarily focuses on **compute-side bottlenecks**, particularly GPU queueing and memory pressure. Its dispatching decisions are largely driven by estimated memory feasibility and expected execution time, while network conditions between agents and LLM instances are not explicitly modeled. In distributed or multi-zone deployments, network latency can become a dominant factor in end-to-end performance, suggesting an opportunity to extend workflow-aware serving frameworks with network-aware resource management.

3. Exposing Application Semantics for End-to-End Optimization

A fundamental obstacle to end-to-end optimization is the limited visibility that serving systems have into application-level structure. Parrot addresses this limitation by arguing that traditional request-based LLM APIs conceal crucial semantic relationships between calls, preventing system-level optimizations across an application’s execution graph [2]. To overcome this, Parrot introduces the concept of **semantic variables**, which explicitly encode the data dependencies between LLM invocations within an application.

By exposing these semantic variables to the serving layer, Parrot enables the system to identify redundant computations, exploit data reuse, and apply pipeline and batching optimizations across related requests. The system demonstrates that making application semantics explicit can significantly improve throughput and latency without requiring changes to the underlying LLM models [2].

While Parrot successfully expands the optimization space by bridging the application–system boundary, its primary emphasis lies in **semantic dataflow exploitation** rather than resource contention management under high load. In particular, Parrot does not explicitly address scenarios where multiple applications or agents compete for shared LLM infrastructure, nor does it consider dynamic network conditions that may influence end-to-end latency in distributed deployments.

4. Fine-Grained Workflow Orchestration for LLM Applications

Complementary to Parrot’s semantic exposure approach, Teola focuses on the **granularity of orchestration** in LLM-based applications [3]. The authors argue that treating entire application modules as indivisible units limits optimization opportunities, especially when workflows contain both LLM and non-LLM components. Teola proposes decomposing applications into **primitive-level dataflow graphs**, enabling more aggressive parallelization and pipeline execution across workflow stages.

Building on this idea, Ayo serves as a system prototype that implements fine-grained orchestration for end-to-end LLM applications, demonstrating practical performance gains in real workloads [4]. These systems highlight the importance of capturing workflow structure at a sufficiently fine granularity to enable meaningful end-to-end optimization.

However, similar to Parrot, Teola and Ayo primarily address **execution structure and scheduling granularity**, with limited attention to resource heterogeneity and dynamic network effects. As LLM applications scale across distributed clusters or geographically separated regions, network latency and bandwidth variability may undermine the benefits of fine-grained orchestration unless explicitly incorporated into the scheduling model.

5. Disaggregating Inference Stages for Goodput-Oriented Serving

In parallel with workflow-level optimizations, significant effort has been devoted to improving the efficiency of LLM inference services themselves. DistServe proposes a novel serving architecture that **disaggregates the prefill and decoding stages** of LLM inference, allowing each stage to be optimized independently under different latency constraints [5]. The motivation behind DistServe is that co-locating prefill and decoding forces a compromise between time-to-first-token (TTFT) and time-per-output-token (TPOT), limiting achievable goodput.

By separating these stages and placing them on different GPU resources, DistServe enables more flexible resource allocation and better alignment with application-level service-level objectives (SLOs). Importantly, DistServe explicitly considers **communication overhead** between

disaggregated stages and leverages network bandwidth characteristics when making placement decisions [5].

DistServe demonstrates that internal inference architecture plays a critical role in end-to-end performance. Nevertheless, its scope is largely confined to the inference service layer. The system does not directly address higher-level multi-agent workflows, nor does it consider how dynamic network conditions might interact with workflow-level scheduling and routing decisions.

6. EWMA-Based Network Latency Estimation and Change Detection

Effective network-aware resource management requires reliable online estimation of network latency. Exponentially Weighted Moving Average (EWMA) techniques provide a well-established foundation for such estimation. Early work by Roberts introduced exponentially weighted averages as a method for detecting small shifts in process behavior [6], while subsequent studies by Lucas and Saccucci analyzed the statistical properties and robustness of EWMA control schemes [7]. In networking systems, EWMA has been widely adopted for smoothing round-trip time (RTT) measurements. Jacobson and Karels applied exponential smoothing to RTT estimation as part of TCP congestion control mechanisms, enabling stable retransmission timeout calculation despite noisy measurements [8]. This approach was later standardized in TCP retransmission timer computation specifications [9].

The appeal of EWMA lies in its simplicity, low computational overhead, and ability to distinguish persistent trends from transient fluctuations. These properties make EWMA particularly suitable for **online network monitoring in distributed systems**, where rapid yet stable responses to changing conditions are required. However, EWMA itself is a building block rather than a complete solution; its effectiveness depends on how smoothed estimates are integrated into higher-level scheduling and routing policies.

7. Summary and Identified Research Gap

Collectively, the reviewed literature demonstrates substantial progress toward end-to-end optimization of LLM-based applications. Workflow-aware serving systems such as Kairos highlight the importance of agent heterogeneity and shared-resource management under load [1]. Parrot and Teola/Ayo emphasize the necessity of exposing application structure and increasing orchestration granularity to unlock end-to-end performance gains [2–4]. DistServe illustrates how inference-stage disaggregation and SLO-aware resource allocation can improve service efficiency [5]. Meanwhile, EWMA-based methods provide a robust foundation for online network latency estimation [6–9].

Despite these advances, a notable gap remains. Existing systems predominantly optimize **compute-side resources and execution structure**, while **network dynamics**—including topology changes, link degradation, and cross-node communication costs—are often treated as secondary considerations or assumed to be static. In distributed multi-agent settings, however, network conditions can significantly influence end-to-end latency, especially under shared infrastructure and dynamic workloads.

This gap motivates further research into **network-aware resource management for multi-agent LLM workflows**, integrating online network latency estimation with workflow-aware scheduling and resource dispatching. Addressing this challenge has the potential to complement existing approaches and provide more robust end-to-end latency guarantees in realistic, dynamic deployment environments.

References

1. Chen J, Shi J, Chen Q, Guo M. *Kairos: Low-latency Multi-Agent Serving with Shared LLMs and Excessive Loads in the Public Cloud*. arXiv preprint [Internet]. 2025 [cited 2026 Jan 11]; arXiv:2508.06948. Available from: <https://arxiv.org/abs/2508.06948>
2. Lin C, Han Z, Zhang C, Yang Y, Yang F, Chen C, et al. *Parrot: Efficient Serving of LLM-based Applications with Semantic Variable*. arXiv preprint [Internet]. 2024 [cited 2026 Jan 11]; arXiv:2405.19888. Available from: <https://arxiv.org/abs/2405.19888>
3. Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. arXiv preprint

[Internet]. 2024 [cited 2026 Jan 11]; arXiv:2401.09670. Available from: <https://arxiv.org/abs/2401.09670>

4. Tan X, Jiang Y, Yang Y, Xu H. *Teola: Towards End-to-End Optimization of LLM-based Applications*. arXiv preprint [Internet]. 2024 [cited 2026 Jan 11]; arXiv:2407.00326. Available from: <https://arxiv.org/abs/2407.00326>
5. Tan X, Jiang Y, Yang Y, Xu H. *Towards End-to-End Optimization of LLM-based Applications with Ayo*. Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25) [Internet]. 2025 [cited 2026 Jan 11];10.1145/3676641.3716278. Available from: <https://doi.org/10.1145/3676641.3716278>
6. Roberts SW. *Control Chart Tests Based on Geometric Moving Averages*. Technometrics [Internet]. 1959 [cited 2026 Jan 11];1(3):239–250. Available from: <https://www.jstor.org/stable/1266443>
7. Lucas JM, Saccucci MS. *Exponentially Weighted Moving Average Control Schemes: Properties and Enhancements*. Technometrics [Internet]. 1990 [cited 2026 Jan 11];32(1):1–12. Available from: <https://www.stat.cmu.edu/technometrics/90-00/vol-32-01/v3201001.pdf>
8. Jacobson V, Karels MJ. *Congestion Avoidance and Control*. In: Proceedings of the ACM SIGCOMM Conference [Internet]. 1988 [cited 2026 Jan 11]. Available from: <https://ee.lbl.gov/papers/congavoid.pdf>
9. Paxson V, Allman M, Chu J, Sargent M. *Computing TCP's Retransmission Timer*. RFC 6298 [Internet]. 2011 [cited 2026 Jan 11]. Available from: <https://www.rfc-editor.org/rfc/rfc6298>

Summary of the Progress of the Project So Far

To date, the project has focused on establishing a solid theoretical and conceptual foundation for subsequent system design and implementation. The primary progress can be summarized in the following aspects.

First, an extensive review of state-of-the-art literature related to multi-agent systems, large language model (LLM) serving, and end-to-end latency optimization has been conducted. Key representative works, including Kairos, Parrot, DistServe, and Teola/Ayo, have been carefully studied to understand current research directions, system architectures, and optimization strategies. Through this process, the project has identified how existing approaches address workflow-aware scheduling, resource sharing under high load, inference-stage optimization, and end-to-end performance considerations, as well as their limitations, particularly with respect to dynamic network conditions.

Second, the project has focused on acquiring and consolidating domain-specific terminology and foundational concepts relevant to this research area. This includes, but is not limited to, concepts such as multi-agent workflows, end-to-end latency, tail latency, service-level objectives (SLOs), inference pipeline stages (prefill and decoding), resource contention, and basic network performance metrics. A clear understanding of these terms has enabled more precise interpretation of prior work and has helped establish a consistent technical vocabulary for subsequent analysis and system design.

Overall, the work completed so far has clarified the research background, identified key challenges in existing systems, and laid the conceptual groundwork necessary for defining a concrete technical approach in the next phase of the project.

是否符合进度? On schedule as per GANTT chart?

YES

下一步 Next steps:

The next phase of the project will focus on transitioning from literature understanding to problem formalization and preliminary system design. The immediate planned tasks are as follows.

First, the project will refine and formalize the specific research problem to be addressed, with particular emphasis on end-to-end latency optimization in multi-agent LLM systems under dynamic network conditions. This includes clearly defining the system model, the scope of network variability considered, and the performance metrics used for evaluation.

Second, based on insights gained from the literature review, an initial system architecture will be designed. This architecture will outline the roles of core components such as workflow tracking, request dispatching, and network-aware decision-making, and will specify how network-related information can be incorporated into the resource management process.

Third, the project will begin preparing the experimental and implementation environment. This includes selecting appropriate tools or frameworks for prototyping, identifying candidate workloads or benchmark scenarios, and planning how network conditions can be monitored or emulated for evaluation purposes.

These steps will provide a concrete foundation for subsequent implementation and experimentation, which will be the focus of later stages of the project.